

60. Trois variables qui parlent de la même chose

Une `struct`, plutôt que `bossX`, `bossY`, `bossVie`.

1. Ce qu'il faut comprendre

Sans `struct`, on écrirait `bossX`, `bossY`, `bossVie` — et le jour où il faut un second ennemi, on recopie trois variables et l'on en oublie une.

Une `struct` fait **au plus 255 octets**, et chaque champ est un octet. C'est un enregistrement, pas un objet : ni héritage, ni constructeur.

La leçon « Une struct : ce qui va ensemble » en fait un usage complet ; celle-ci n'en montre que la forme.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1336 octets** de cartouche.

1. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {
    uint8_t x;
    uint8_t y;
    uint8_t vie;
};
```

2. UNE VARIABLE — BOSS

```
Ennemi boss;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    boss.x = 40;
    boss.y = 60;
    boss.vie = 3;

    texte(2, 4, "X");   nombre(6, 4, boss.x);
    texte(2, 6, "VIE"); nombre(6, 6, boss.vie, 1);

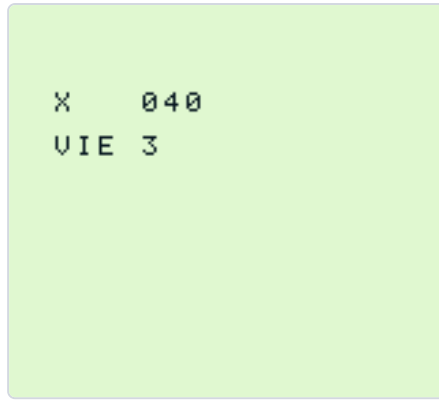
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« X 040 » et « VIE 3 » : les deux champs de la même struct, relus.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le champ `x` est écrit et relu
- ✓ le champ `vie` aussi
- ✓ les trois champs tiennent en trois octets

Ensuite : **61. Un tableau de « struct »** — niveau 8 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++